



Ink Token Upgradeability

Security Assessment (Summary Report)

August 22, 2025

Prepared for:

Ink Foundation

Prepared by: **Samuel Moelius**

Table of Contents

Table of Contents	1
Project Summary	2
Project Targets	3
Executive Summary	4
A. Non-Security-Related Recommendations	6
About Trail of Bits	9
Notices and Remarks	10

Project Summary

Contact Information

The following project manager was associated with this project:

Samuel Greenup, Project Manager
sam.greenup@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Samuel Moelius, Consultant
samuel.moelius@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
August 7, 2025	Pre-project kickoff call
August 15, 2025	Delivery of report draft
August 22, 2025	Delivery of final summary report

Project Targets

The engagement involved reviewing and testing the following target.

ink-token

Repository	https://github.com/inkfoundation/inktoken
Version	d5349f0c6fab037dbcb71a7548100157ba269b02
Previously Audited Version	e96e7f3330a12063e2aa88b380415a3c5e3d3a5b
Type	Solidity
Platform	Ethereum

Executive Summary

Engagement Overview

Ink Foundation engaged Trail of Bits to review the security of changes made to Ink Token to enable upgradeability.

A team of one consultant conducted the review from August 14 to August 15, 2025, for a total of two engineer-days of effort. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

Ink Token uses OpenZeppelin's `contracts-upgradeable` to enable upgradeability. The token uses the latest major version of those contracts (version 5), which implies the use of `ERC-7201: Namespaced Storage Layout`. The implementation is straightforward and relatively well tested. We found no vulnerabilities in the implementation.

We verified the following aspects of the implementation:

- It adheres to the guidelines in `Writing Upgradeable Contracts`.
- It uses the OpenZeppelin `Upgrades` plugin.
- Removing the `_authorizeUpgradeable` function causes InkToken to no longer build.
- ERC-7201 hashes are correct for both the `main token` and the `test token`.
- InkToken's initializer calls all non-empty parent initializers.
- Removing calls to non-empty parent initializers causes at least one test to fail (specifically, the `test_upgrade` test).
- Sequential upgrades work, e.g., an upgrade to V2 of the token followed by an upgrade to V3.
- `test_upgrade` works with a token that does not inherit from InkToken.

We also reviewed the following:

- The `Deploy.s.sol` and `InitialSetup.s.sol` scripts to ensure proper token interaction
- Tests to ensure that important conditions are checked
- Warnings produced when `Slither` is run over the code

These actions did not yield any findings. We do provide some non-security-related recommendations in [appendix A](#), but these do not mitigate vulnerabilities.

A. Non-Security-Related Recommendations

The following recommendations are not associated with specific vulnerabilities. However, implementing them may enhance code readability and prevent the introduction of vulnerabilities in the future.

- **Correct the typo in figure A.1, i.e., change “toke” to “token.”** Also, consider changing “deploy” to “upgrade.”

```
56 // deploy toke contract
57 Upgrades.upgradeProxy(
```

Figure A.1: Comment with a typo (“toke” should be “token”) ([inktoken/script/MockUpgrade.s.sol#56–57](#))

- **Remove the ERC1967Proxy import from Deploy.s.sol, as the import is unused.**

```
7 import {ERC1967Proxy} from
"@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
```

Figure A.2: Unused import in Deploy.s.sol ([inktoken/script/Deploy.s.sol#7](#))

- **Ensure that CI is being run.** The repo’s Actions page says “0 workflow runs.” Also, the `test.yml` file does not set up a `.env` file, and the tests fail if the `PRIVATE_KEY` environment variable is empty.
- **Run a code formatter over the code.** The code does not appear to be formatted at present. For example, in figure A.3, the line beginning with `vm.startBroadcast` is not sufficiently indented.

```
64 console2.log("mint amount / decimals: ", mint_amount / (10 **
token.decimals()));
65
66
67 vm.startBroadcast(vm.envUint("PRIVATE_KEY"));
68 // blacklist each address in the blacklist
69 for (uint256 i = 0; i < config.blacklist.length; i++) {
```

Figure A.3: Excerpt of InitialSetup.s.sol ([inktoken/script/InitialSetup.s.sol#64–69](#))

- **Test sequential upgrades.** Currently, the tests perform exactly one upgrade (figure A.4). Performing two or more upgrades will provide additional assurance that the upgrade process works correctly.

```

203 // Upgrade the contract
204 vm.startPrank(contractOwner);
205 Upgrades.upgradeProxy(
206     address(token),
207     "TestUpgradeERC20.sol",
208     abi.encodeCall(TestUpgradeERC20.initializeV2, ()))
209 );
210 vm.stopPrank();
211 TestUpgradeERC20 upgradedToken = TestUpgradeERC20(address(token));

```

Figure A.4: Excerpt of `InkToken.t.sol` where the contract is upgraded ([inktoken/test/InkToken.t.sol#203–211](#))

- **Test with an upgrade contract that does not inherit from the existing `InkToken` contract.** Inheriting from the existing contract seems to interfere with OpenZeppelin’s Upgrades plugin. For example, adding a `@custom:oz-upgrades-validate-as-initializer` annotation to `initializeV2` causes `test_upgrade` to fail (A.5). Because the function lacks this annotation, it is not validated to ensure that all parent initializers are called (A.6).

```

33 function initializeV2() public reinitializer(2) {
34     TestupgradeStorage storage $ = _getTestupgradeStorage();
35     $.is_upgraded = true;
36 }

```

Figure A.5: Adding a `@custom:oz-upgrades-validate-as-initializer` annotation to this function causes `test_upgrade` to fail ([inktoken/test/TestUpgradeERC20.sol#33–36](#))

```

49 /// @custom:oz-upgrades-validate-as-initializer
50 function initialize(string memory _name, string memory _symbol) public
reinitializer(1) {
51     __ERC20_init_unchained(_name, _symbol);
52     __ERC20Votes_init_unchained();
53     __EIP712_init_unchained(_name, "1");
54     __Ownable_init(msg.sender);
55     InkTokenStorage storage $ = _getInkTokenStorage();
56     $.isTransferPaused = true;
57 }

```

Figure A.6: `initializeV2` from figure A.5 should call the initializers highlighted in this figure. However, because `initializeV2` lacks a `@custom:oz-upgrades-validate-as-initializer` annotation, the Upgrades plugin does not warn about this. ([inktoken/src/InkToken.sol#49–57](#))

- **Add calls to all of the parent contracts’ initializers.** Calls to the following initializers are currently missing. The initializers are empty, so there is no security impact. However, one would have to rely on the Upgrades plugin to warn about them if they become non-empty. Adding calls to them now saves having to rely on the Upgrades plugin.

- `__Context_init`
- `__Nonces_init`
- `__Votes_init`

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2025 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Ink Foundation under the terms of the project statement of work and has been made public at Ink Foundation's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

All activities undertaken by Trail of Bits in association with this project were performed in accordance with a statement of work and agreed upon project plan.

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test the controls and security properties of software. These techniques augment our manual security review work, but each has its limitations: for example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. Their use is also limited by the time and resource constraints of a project.